UNIT-4

DATA STRUCTURE:

The data structure can be defined as the collection of elements and all the possible operations which are required for those set of elements. In other words data structures will tell us specific set of elements and corresponding set of operations.

The data structures can be implemented by building the suitable algorithms for them.

For example: consider a set of elements that can be stored in array. Various operations such as reading the elements and storing them at appropriate index can be performed. If we want to access any particular element then that element can be retrieved from the array. Thus reading, searching, writing would be the operations required to perform these tasks for the elements. Here array is called as a data structures because the basic data objects in array are the integer elements and the operations that are performed are reading, writing and searching of elements. This combination of data object and elementary operations together form a data structure called array.

The logical (or) mathematical model of a particular data organization is called as **DATA STRUCTURE. (or) arranging the data in a sequential order (or) random order is known as DS.**

Primitive data structure: The data structure, which is available in the compiler, is known as a *primitive data structure*.

Non-primitive data structure: The data structure, which is not available in the compiler, is known as *non-primitive data structure*.

Linear Data Structure: The data structure in which each element has access to maximum of one predecessor element and maximum of one successor element is known as *linear data structure*. **Example:** Stack, Queue, etc.

Non-linear Data Structure: The data structure in which each element can access any number of predecessor elements and any number of successor elements is known as *Non-linear data structure*. **Example:** Tree, Graphs, etc.

Static Data Structure: The data structure in which the number of elements is fixed, is known as *Static Data Structure*. **Example:** Arrays.
Dynamic Data Structure: The data structure in which the number of elements is not fixed, is known as *Dynamic Data Structure*. **Example:** Linked List.

TYPES OF DATA STRUCTURE:

DATA STRUCTURE

PRIMITIVE DATA TYPES

a. **CHARACTER**

b. **INTEGER**

c. **DOUBLE**

d. **FLOATING POINT**

NON-PRIMITIVE DATA TYPES

a. **ARRAYS**

b. **FILES**

c. **LINKED LIST**

- **i. LINEAR DATA STRUCTURE**
  1. **STACKS**
  2. **QUEUES**
- **ii. NON-LINEAR DATA STRUCTURE**
  1. **TREES**
  2. **GRAPHS**

ABSTRACT DATA TYPE (ADT):

A data structure along with all the possible operations on that data structure is called as an **abstract data type**.
An ADT can be implemented in the hardware/software of a computer.
For instance, a linked list is a data structure and all the possible operations on the linked list, along with the linked list representation itself is an ADT.

Abstract Data Type (ADT)

Value Definition

a. **Defintion Clause**

b. **Condition Clause**

Operation Definition

a. **Function**

b. **Pre Condition**

Post Condition

Let us consider the representation of integer data type as an ADT. We will consider only two operations addition and division**.**

AbstractDataType arithmetic

{

Value Definition

Definition Clause: the values must be in between the minimum and maximum values specified for the particular computer.

Condition Clause: values should not include decimal point.

Operations

i) add(a,b)

Function: add the two integers a and b.

Precondition: no precondition.

Postcondition : output =a+b.

ii) div(a,b)

Function: divide the two integers a by b.

Precondition: b!=0.

Postcondition: output =a/b.

}

ARRAYS (STATIC TYPE)

It is a collection of similar data items under a single variable name of a single data type is known as **array.**

Arrays offer various applications. For example,

- Matrix Multiplication /Matrix Addition/ Matrix Subtraction
- Sparse Matrix
- Records
- Order List

Sparse Matrix: A matrix is a representation of elements in rows and columns. Sparse Matrix is a matrix with a high proportion of zero entries. The following fig-a shows the matrix and sparse matrix.
The most commonly used n square sparse matrix in various applications.

3 2 1 5	4 0 0 0	1	5 2
1 0 4 2	0 0 0 0	2 3	1 3 4

0 6 5 6	0 0 3 0	4 5 6	7 3 2
2 1 0 5	0 2 0 0	7 8 9 10	4 6 5
a. Matrix	b. sparse matrix	c.Triangular Matrix	d. Tridiagonal Matrix

A matrix requires a lot of space in memory. A two-dimensional array is used to represent matrixs. Thus, a 1000*1000 matrix requires huge storage locations in memory. If it is a sparse matrix, to store non-zero elements a huge amount of memory will be wasted. A two-dimensional array may not be suitable for sparse matrixs.

Suppose to store a triangular matrix in memory, it is clear that storing all entries above the diagonal wastes the memory since they are all zeros. In such a case using a triple representation (i,j,value) can save the memory to represent non-zero elements of sparse matrix.

AbstractDataType Array
{
Instances: An array A of some size, index I and total number of elements in the array n.
Operations:
store() – This operation stores the desired elements at each successive location.
display() – this operation displays the elements of the array.
}

ADT OPERTIONS FOR ARRAY

- Creating a Array
- Inserting an element into the Array (First, Middle, Last)
- Deleting an element from the Array (First, Middle, Last)
- Traverse the Array
- Searching an element in the Array
- Sorting an element in the Array
- Reversing an element in the Array
- Merging the Two Array
- Counting the elements in the Array

LINKED LISTS (DYNAMIC TYPE)

Self-Referential-Structure: A structure definition which includes at least one member as a pointer to the same structure is known as Self-referential structure.

E:

```
struct student

{

char name[50];

int rno;

struct student *ptr;
```

```
};
```

Self-referential structure are mainly used in the implementation of linked data structures like lists, trees etc.

Consider an implementation of linked-lists. A linked list is a collection of nodes, where each node points to the next node in the list.

The advantage of using linked lists is that the data can be added or deleted easily.

In a linked list, each node consists of two items , one containing the data and the other containing the address of the next element.

E:

```
struct node

{

char name[50];

int rno;

struct student *ptr;

};
```

The structure declared is of type node. It consists of three members. The first member is an char type i.e., name and the second member is an int type i..e, rno and the third member is a pointer to the next node in the list, which is shown below,

node

In a linked list, nodes represent self-referential structures and the arrows represent pointers that make a linked list a self-referential structures.

TYPES OF LINKED LIST

- **Singly linked list:** each node contains data and a pointer to the next node. The last node points to NULL indicating the end of the linked list.
- **Circularly Singly linked list:** similar to a singly linked list, but the last node points to the first node of the list.
- **Doubly linked list:** each node contains data and two pointers: one points to the previous node and one points to the next node. By convention the two pointers are called *prev and *next. *prev pointer of the first node points to NULL,

indicating the beginning of the list, and the *next pointer of the last node points to NULL, indicating the end of the list.

- **Circularly doubly linked list:** similar to a doubly linked list, but the *prev pointer of the first node points to the last node of the list, and the *next pointer of the last node points to the first node of the list thus completing a circle.

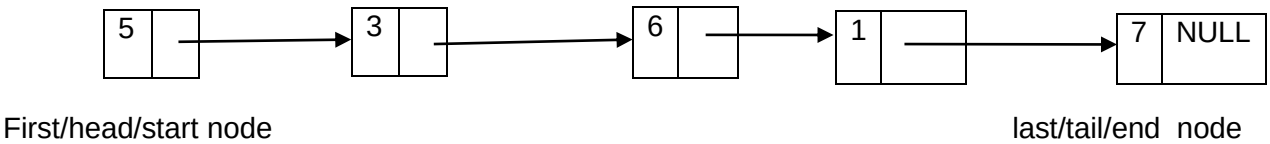


Fig-a: **Singly Linked List**

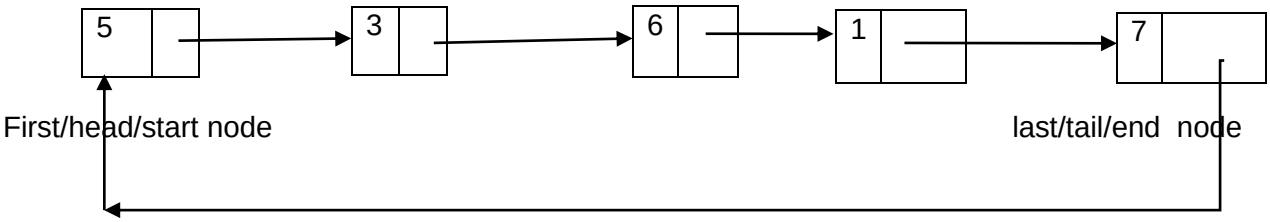


Fig-b: **Circular Singly Linked List**

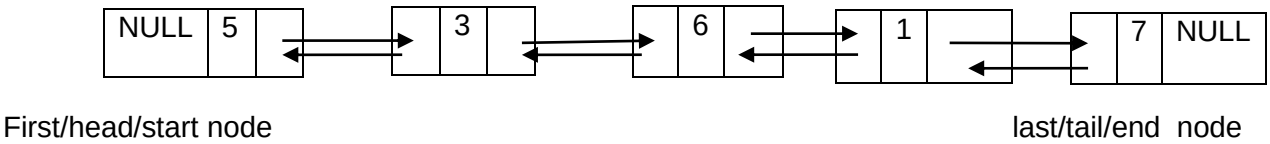


Fig-c: **Doubly Linked List**

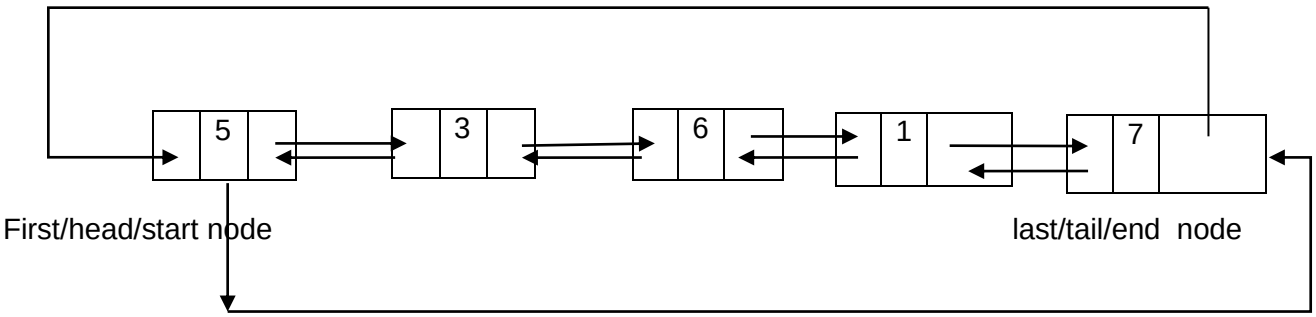


Fig-d: **Circular Doubly Linked List**

AbstractDataType List
{
Instances:
Data_type arrayname[max_size];
Data_type array_index;
The list is created in the array.
Precondition:
The array_index should be within 0 to max_size of array.
Operations:
create() – The list is created using an array. That means the elements are stored sequentially in an array.
display() – each element of the array is displayed sequentially.
Insert() – the element is inserted in the array at the location specified by array_index.
Delete() – the element specified by the array_index is deleted from the array.
}

ADT OPERTIONS FOR LIST

- Creating a List
- Inserting an element into the List (First, Middle, Last)
- Deleting an element from the List (First, Middle, Last)
- Traverse the List
- Searching an element in the List
- Sorting an element in the List
- Reversing an element in the List
- Merging the Two List
- Counting the elements in the List

Linked lists offer various applications. For example,

- They form the basis of various other data structures such as stacks, queues and trees, graphs.
- Dynamic memory stroage
- They are used in various gaming applications.
- They are used to implement internals of file system in various operating systems.
- They offer a simple and convenient mechanism to perform various arithmetic operations on polynomial expressions/ Polynomial manipulation
- Symbol table
- Sparse matrix representation

<u>COMPARE ARRAYS AND LINKED LISTS</u>

1. Insertion and Deletion in a linked list is fast as compared to arrays.
2. However, accessing elements is faster in arrays as compared to linked lists.
3. Arrays can retrieve the elements both the sequential and random accessed methods, that are presented/stored in that array variable.
   It is static type memory allocation.

4. Linked List can retrieve the elements only by through the sequential accessed method, that are presented/stored in that Linked list variable.
   It is dynamic type memory allocation.

- Arrays allow random access to the nth element of the array. However, arrays require one block allocation, meaning the addresses of the elements are in sequential order in the memory pool. If the memory pool only has 10 blocks of free memory, allocating an array that requires 11 blocks of memory will be impossible. Arrays are of fixed size, inserting additional data into the array would mean cloning the existing array and porting it over to a new array with a new size. Removing elements from the array would require shifting remaining elements down upon resetting the target element, and placing the NULL element at the end.
- Linked lists do not allow random access – accessing the nth element of the list will require iterating through the list of nodes from the first node. However, insertion and deletion of data is easy since nodes are coupled with pointers. Size of linked lists can be modified dynamically, and since nodes are connected with pointers, the memory location of each node does not have to be adjacent to each other.

UNIT-3.1

STACKS

A stack is a collection of data items that can be accessed at only one end, called top. The last item inserted in a stack is the first one to be deleted.
A stack is called a LIFO data structure.
There are two operations that can be performed on stacks. they are:
PUSH
POP
stacks can be implemented by using both arrays and linked lists.

STACKS ARE USED IN MANY APPLICATIONS. SOME OF THE APPLICATION DOMAINS OF STACKS ARE AS FOLLOWS:

- implementing function calls/recursion calls
- maintaining the UNDO list for an application
- conversion of number bases
- checking the nesting of parenthesis in an expression /balancing of the matching parenthesis
- evaluating expressions/expression evaluation

AbstractDataType Stack
{
Instances:
Data_type stack[max_size];
Data_type top;
Precondition:
The top should not exceed the limit of max_size.
Operations:
create() – The stack is created as the structure for stack is declared.
display() – each element of the stack is displayed sequentially from top to bottom.

push() – the element is pushed onto the stack. The recently pushed element becomes the topmost element of the stack.
pop() – always the topmost element is deleted from the stack.
peep() – This function returns the topmost element on the *stack* but does not remove it from the *stack.*
stfull() – this is the condition that is to be checked before performing the push operation. This function tells whether the stack is full (or) not.
stempty() - this is the condition that is to be checked before performing the pop operation. This function tells whether the stack is empty (or) not.
}

ADT OPERTIONS FOR STACK

- Creating a Stack
- Pushing/Inserting an element in the Stack
- Popping/Deleting an element from the Stack
- Traverse the Stack

PRINCIPLES OF STACK

- LIFO – Last Inserted Element First Deleted Out
- FILO - First Inserted Element Last Deleted Out

IMPLEMENTATION OF STACK

- stack using arrays (initially top=-1)
- stack using linked lists (initially top=NULL)

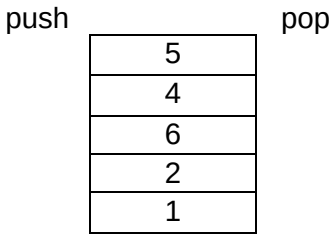
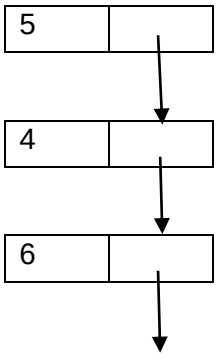


Fig-a: **Stack using Arrays**



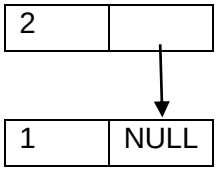


Fig-b: **Stack using Linked List**

STACK APPLICATIONS

Expressions

INFIX (operand1 operator operand2)	**PREFIX (operator operand1 operand2)**	**POSTFIX (operand1 operand2 operator)**
X	X	X
X+Y	+XY	XY+
(X+Y)	+XY	XY+
X+Y+Z	++XYZ	XY+Z+
X+Y*Z	+X*YZ	XYZ*+
(X+Y)*Z	*+XYZ	XY+Z*
(X+Y)+Z	+XY+Z	XYZ++
X+(Y+Z)	+X+YZ	XYZ++
X/(Y+Z)	/X+YZ	XYZ+/
X*Y*Z	**XYZ	XY*Z*
(X+Y)*(P+Q)	*+XY+PQ	XY+PQ+*
(X+Y)*(P-Q)	*+XY-PQ	XY+PQ-*
X/Y*Z	*/XYZ	XY/Z*
(A+B/E)*(D+F)	*/+ABE+DF	AB+E/DF+*
A+(B+C)/D	+/A*BCD	ABC*D/+
(A+B)*(C/D)	*+AB/CD	AB+CD/*

ALGORITHM FOR CONVERSION FROM INFIX TO POSTFIX EXPRESSION FORM

1. Read the expression from left to right.
2. If the input symbol read is **'('** then push it into the stack.
3. If the input symbol read is an operand then place it in postfix expression.
4. If the input symbol read is an operator then
    a. Check if the precedence of the operator which is in the stack has greater precedence than the precedence of the operator read, if so then remove that

symbol from stack and place it in the postfix expression. Repeat **step (4.a)** till you get the operator in the stack has greater precedence than the operator being read.

b. Otherwise push the operator being read onto the stack.

5. If the input symbol read is a closing parenthesis **')'** then pop all the operators from the stack, place them in postfix expression till the opening parenthesis is not popped. The **'('** should not be place in the postfix expression.

6. Finally print the post fix expression.

Example 1: A*B+C $

INPUT	**STACK**	**OUTPUT**
None	Empty	None
A	Empty	A
*	*	A
B	*	AB
+	POP * and PUSH	AB*
C	+	AB*C
$	Empty	AB*C+

Example 2: (A+B)*(C-D) $

INPUT	**STACK**	**OUTPUT**
None	Empty	None
(	(	
A	(	A
+	+ **(PUSH +)** (	A
B	+ (	AB

)	) **(when closing parenthesis comes pop everything and print except '(')** + (	AB+
*	*	AB+
(	(**(then '(' because '(' ha more priority than '*')** *	
C	(*	AB+C
-	- (*	AB+C
D	- (*	AB+CD
)	) **(POPPING -)** - (*	AB+CD-
$	*	AB+CD-*

ALGORITHM FOR EVALUATION OF POSTFIX EXPRESSION FORM

1. Read the postfix expression from left to right.
2. If the input symbol read is an operand then push it onto the stack.
3. It the operator is read POP two operands and perform arithmetic operations if operator is
    a. + then result = operand1 + operand2
    b. - then result = operand1 - operand2
    c. * then result = operand1 * operand2
    d. / then result = operand1 / operand2
4. Push the result onto the stack.
5. Repeat **steps 1-4** till the postfix expression is not over.

Example 1: 9-((3*4)+8)/4

934*8+4/-

INPUT	**STACK**	**OUTPUT**
None	Empty	None
9	9	
3	3 9	
4	4 3 9	
*	**(pop two operands & perform multiplication of 3 and 4. then push the result into the stack i.e, 3*4=12)** 12	

	9	
8	8 12 9	
+	**(pop two operands & perform addition of 12 and 8. then push the result into the stack i.e, 12+8=20)** 20 9	
4	4 20 9	
/	**(pop two operands & perform division of 20 and 4. then push the result into the stack i.e, 20/4=5)** 5 9	
-	**(pop two operands & perform subtraction of 9 and 5. then push the result into the stack i.e, 9-5=4)** 4	4

Example 2: AB+C-BA+C ^- $ (for A=1, B=2, C=3)

INPUT	STACK	OUTPUT
None	Empty	None

1	1	
2	2 1	
+	**(pop two operands & perform addition of 1 and 2. then push the result into the stack i.e, 1+2=3)** 3	
3	3 3	
-	**(pop two operands & perform subtraction of 3 and 3. then push the result into the stack i.e,3-3=0)** 0	
2	2 0	
1	1 2 0	
+	**(pop two operands & perform addition of 1 and 2. then push the result into the stack i.e, 1+2=3)** 3 0	
3	3 3 0	

^	**(pop two operands & perform exponention of 3 and 3. then push the result into the stack i.e, 3^3=27)** 27 0	
-	**(pop two operands & perform subtraction of 0 and 27. then push the result into the stack i.e, 0-27=-27)** -27	
$	-27	-27

UNIT-3.2

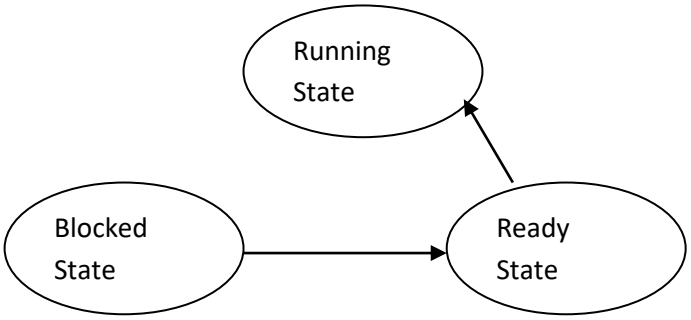
QUEUES/LINEAR QUEUES

A queue is a linear data structure in which the elements are inserted at the one end and deleted from the another end.
The last item inserted in a queue is the last one to be deleted.
A queue is called a LILO data structure.
There are two operations that can be performed on queue. they are:
REAR / INSERT
FRONT/REMOVE
queues can be implemented by using both arrays and linked lists.
A queue implemented in the form of a circular array overcomes the problem of unutilized space encountered in queues implemented bt using a linear array.
A queue implemented by using a linked list is called a linked queue.
QUEUES ARE USED IN MANY APPLICATIONS. SOME OF THE APPLICATION DOMAINS OF QUEUES ARE AS FOLLOWS:

- printer spooling
- job/CPU scheduling
- mail service
- keyboard buffering
- Elevator
- Categorizing data
- Simulation of time-sharing system
- Round Robin

cpu/job scheduling

In operating system various programs are getting executed.
We will call these programs as jobs. In this process, some programs are in executing state. The state of these programs is called as ‘running’ state. Some programs which are not executing but they are in a position to get executed at any time such programs are in the ‘ready’ state. And there are certain programs which are neither in running state nor in ready state. Such programs are in a state called as blocked state.
The operating system maintains a queue of all such running state, ready state, blocked state programs. Thus use of queues help the operating system to schedule the jobs. The jobs which are in running state are removed after complete execution of each job, then the jobs which are in ready state change their state from ready to running and get entered in the queue for running state. Similarly the jobs which are in blocked state can change their state from blocked to ready state. These jobs then can be entered in the queue for ready state jobs. Thus every job changes its state and finally gets executed. See the fig-a.



Jobs are changing their state of execution

queue for running state jobs

J1	J2	J3	J4	r1				
0	1	2	3	4	5	6	7	8

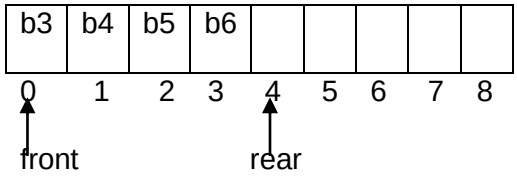
front rear

queue for ready state jobs

r2	r3	r4	r5	r6	r7	b1	b2	
0	1	2	3	4	5	6	7	8

front rear

queue for blocked state jobs



Thus queues are effectively used in operating system for scheduling the jobs.

Fig-a: **Job/CPU Scheduling in Operating System**

AbstractDataType Queue
{
Instances:
Data_type queue[max_size];
Data_type front, rear;
Precondition:
The front and rear should be within the max_size.
Operations:
create() – The queue is created as the structure for queue is declared.
display() – each element of the queue is displayed sequentially from front to rear.
insert()/ENQUEUE – the element can be inserted in the queue by rear end.
delet()/DEQUEUE – the element at front end deleted each time.
quefull() – this is the condition that is to be checked before performing the insertion operation. This function tells whether the queue is full (or) not.
queempty() - this is the condition that is to be checked before performing the deletion operation. This function tells whether the queue is empty (or) not.
}

ADT OPERTIONS FOR QUEUES

- Creating a Queue
- Rearing/Inserting an element in the Queue
- Fronting/Deleting an element from the Queue
- Traverse the Queue

PRINCIPLES OF QUEUES

- FIFO – First Inserted Element First Deleted Out
- LILO - Last Inserted Element Last Deleted Out

IMPLEMENTATION OF QUEUES

- queue using arrays (initially font=rear=-1)
- queue using linked lists

- (initially front=rear=NULL)

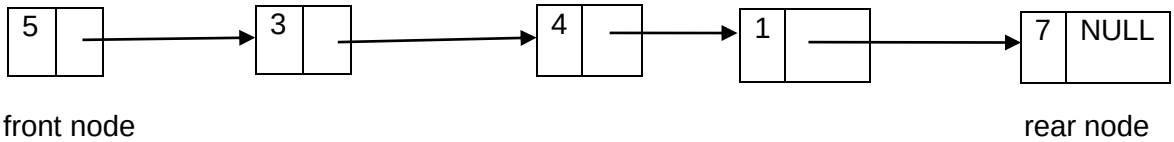


Fig-a: **Queue using Linked List**

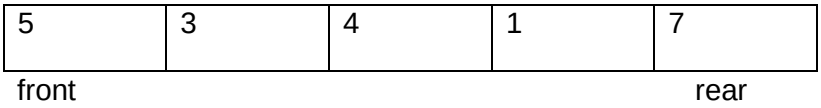


Fig-b: **Queue using Arrays**

TYPES OF QUEUES
There are four types of queues

- ***Linear Queue***
- ***Circular Queue***
- ***DoubleEnded Queue(Dequeue)***
- ***Priority Queue***

Circular Queue

As we have seen, in case of linear queue the elements gets deleted logically. This can be shown by following fig-a;

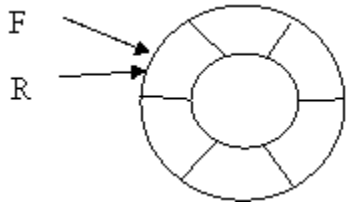


Fig-a: Circular Queue

We assume circularqueuesize is 6, the q[0]=10, q[1]=20, q[2]=30, q[3]=40, q[4]=50 and q[5]=60.

We have deleted the elements 10,20 and 30 means simply the front pointer is shifted ahead. We will consider a queue from front to rear always. And now if we try to insert any more element

then it won't be possible as it is going to give "queue full" message. Although there is a space of elements 10,20 and 30 (these are deleted elements), we cannot utilize them because queue is nothing but a linear array.

Hence there is a concept called circular queue. The main advantage of circular queue is we can utilize the space of the queue fully. The circular queue is shown by fig-a.

Considering that the elements deleted are 10,20 and 30.

There is a formula which has to be applied for setting the front and rear pointers for a circular queue.

rear = (rear+1)%sizeofthequeue

front = (front+1)%sizeofthequeue

rear = (rear+1)%sizeofthequeue

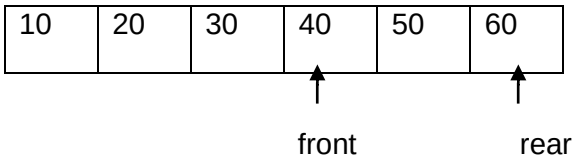
rear=(5+1)%6=6%6

rear=0 so, we can store the element 70 at 0th location similarly while deleting the element.

front = (front+1)%sizeofthequeue

front=(3+1)%6=4%6

front=4



so delete the element 4th location i.e, element 50

DoubleEndedQueue (Dequeue)

In a linear queue, the usual practice is for insertion of elements we use one end called rear and for deletion of elements we use another end called as front. But in the doubly ended queue we can make use of both the ends for insertion of the elements as well as we can use both the ends for deletion of the elements. That means it is possible to insert the elements by rear as well as by front. Similarly it is possible to delete the elements from front as well as from rear. Just se the following fig-a to understand the concept of doubly ended queue i.e, dequeue.

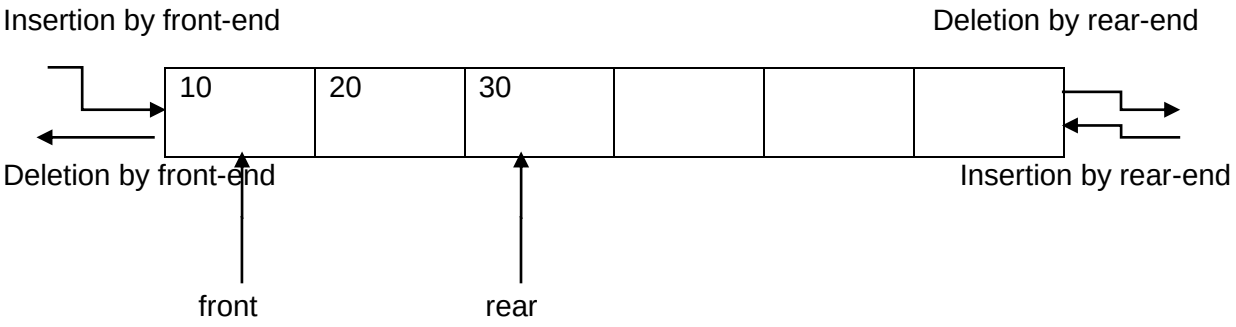
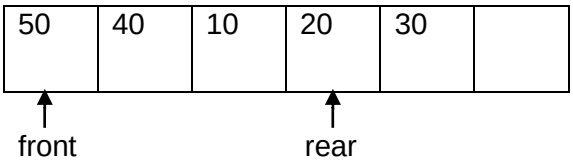


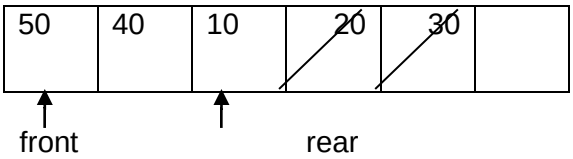
Fig-a : **doubly ended queue (dequeue)**

As we know, normally we insert the elements by rear end and delete the elements from front end. Let us say we have inserted the elements 10, 20, 30 by rear end as shown in the above fig-a. Now if we wish to insert any element from front end then first we have to shift all the elements to the right.

For example, if we want to insert 40 by front end then, the dequeue will be

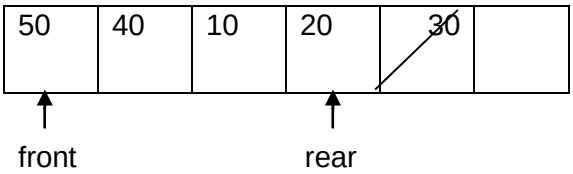


Inserted 40 in the queue by front end



Inserted 50 in the queue by front end

Fig-b: **Insertion by front end**



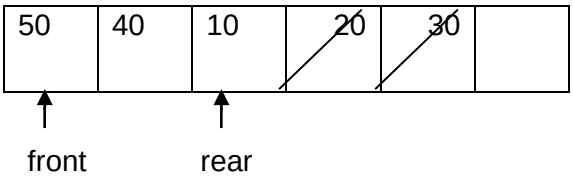


Fig-c: **Deletion by rear end**

We can place -1 for the element which has to be deleted.

<u>Priority Queue</u>

The Priority queue is a data structure having a collection of elements which are associated with specific ordering (or) priorities.

There are two types of priority queue

Ascending priority queue: It is a collection of items in which the items can be inserted arbitrarily but only smallest element can be removed.

Descending priority queue: It is a collection of items in which the items can be inserted arbitrarily but only largest element can be removed.

In priority queue, the elements are arranged in any order and out of which only the smallest (or) largest allowed to delete each time.

The implementation of priority queue can be done using arrays (or) linked list. The data structure heap is used to implement the priority queue effectively.

AbstractDataType PriorityQueue
{
Instances:
Data_type pr_queue[max_size];
Data_type front, rear;
Precondition:
The front and rear should be within the max_size.
Operations:
create() – The queue is created as the structure for queue is declared.
display() – each element of the queue is displayed sequentially from front to rear.
insert() – the element can be inserted in the queue by rear end.
delet() – the element at front end deleted each time. If the priority queue is ascending priority queue then only smallest element is deleted each time. And if the priority queue is descending priority queue then only largest element is deleted each time.
pr_quefull() – this is the condition that is to be checked before performing the insertion operation. This function tells whether the queue is full (or) not.
pr_queempty() - this is the condition that is to be checked before performing the deletion operation. This function tells whether the queue is empty (or) not.
}

ADT OPERTIONS FOR PRIORITY QUEUES

- Creating a Queue
- Rearing/Inserting an element in the Queue
- Fronting/Deleting an element from the Queue
- Traverse the Queue

IMPLEMENTATION OF PRIORITY QUEUES

- Priority queue using arrays(Sorted List (or) Unsorted List)
- Priority queue using linked lists (Sorted List (or) Unsorted List)
- Priority queue using Binary Search Tree (BST)

- Priority queue using Heap

APPLICATION OF PRIORITY QUEUES

- Scheduling of jobs in operating system
- Car rental services
- Event list
- Job scheduling
- Allocating CPU resources to the most urgent task
- Communications
- Send the most urgent message first
- Event-driven simulation
- Printer queue
- Sorting-heap sort
- Shortest path graph algorithms (greedy algorithm)

Scheduling of jobs in operating system: A typical application of priority queue is scheduling the jobs in operating system. Typically operating system allocates priority to the jobs. The jobs are placed in the queue and position of the job in queue determined their priorities. In operating system there are three kinds of jobs. These are real time jobs, foreground jobs and background jobs. The operating system always schedules the real time jobs first, then if there is no real time job pending then it schedules for the foreground jobs, and finally it schedules the background jobs.

Car rental services: A Car rental services is a kind of service which allows the user to use the car on rental basis. User can use the car by paying the charge on per hour basis. There is fixed amount of charge per hour. To implement this application, a priority queue of users is maintained. When user requests for the car his request is kept in the priority queue. The user who wants to use the car for minimum amount is always selected first when the car is available. Thus a min queue is maintained for the users. To gain the maximum profit, the car must not be kept idle. If all the users are demanding for same amount of time to use the car service, then the user who is ready to pay more will be selected first, thus a max queue is maintained.

Round Robin Algorithm: Round Robin is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way.

- It is simple, easy to implement, and starvation-free as all processes get fair share of CPU.

- One of the most commonly used technique in CPU scheduling as a core.
- It is preemptive as processes are assigned CPU only for a fixed slice of time at most.
- The disadvantage of it is more overhead of context switching.

Illustration:

Round Robin Example:

Process	Duration	Order	Arrival Time
P1	3	1	0
P2	4	2	0
P3	3	3	0

Suppose time quantum is 1 unit.

P1	P2	P3	P1	P2	P3	P1	P2	P3	P2

0 10

P1 waiting time : 4

P2 waiting time: 6

P3 waiting time: 6

The average waiting time(AWT) : (4+6+6)/3=5.33

How to compute below times in Round Robin using a program?

1. Completion Time: Time at which process completes its execution.
2. Turn Around Time: Time Difference between completion time and arrival time. Turn Around Time = Completion Time – Arrival Time
3. Waiting Time(W.T): Time Difference between turn around time and burst time. Waiting Time = Turn Around Time – Burst Time

We assumed arrival times as 0, so turn around and completion times are same.

The tricky part is to compute waiting times. Once waiting times are computed, turn around times can be quickly computed.

Steps to find waiting times of all processes:

```
1- Create an array rem_bt[] to keep track of remaining burst time of processes. This
array is initially a copy of bt[] (burst times array)
2- Create another array wt[] to store waiting times of processes. Initialize this array as 0.
3- Initialize time : t = 0
4- Keep traversing the all processes while all processes are not done. Do following for
i'th process if it is not done yet.
   a- If rem_bt[i] > quantum
      (i)  t = t + quantum
      (ii) bt_rem[i] -= quantum;
   b- Else // Last cycle for this process
      (i)  t = t + bt_rem[i];
      (ii) wt[i] = t - bt[i]
      (ii) bt_rem[i] = 0; // This process is over
```

Once we have waiting times, we can compute turn around time tat[i] of a process as sum of waiting and burst times, i.e., wt[i] + bt[i]

UNIT-3 PROGRAMS

SNO	STACKS	QUEUES
1	Write a C program that uses functions to implement Stack (its operations) using Arrays.	Write a C program that uses functions to implement Queue (its operations) using Arrays.
2	Write a C program that uses functions to convert a given infix expression into postfix expression and infix expression into prefix expression, implement stack using array	Write a C program that uses functions to implement a circular queue using arrays
3	Write a C program that uses functions to convert a given postfix expression into infix expression and postfix expression into prefix expression, prefix expression into infix expression and prefix expression into postfix expression, implement stack using array	Write a C program that uses functions to implement a priority queue using arrays
4	Write a C program that uses functions to evaluate the prefix expression, implement stack using array	Write a C program that uses functions to implement a double ended queue using arrays.
5	Write a C program that uses functions to evaluate the postfix expression, implement stack using array	Write a C program that uses functions to implement a input restricted double ended queue and output restricted double ended queue using arrays (or) pointers/double linked list
6	Write a C program that uses functions to reversing the list of elements, implement stack using array	Write a C program that uses functions to find the round robin algorithm, implement queue using array
7	Write a C program that uses functions to find the factorial calculation, implement stack using array	
8	Write a C program that uses functions to reversing the string, implement stack using array	
9	Write a C program that uses functions to implement Stack (its operations) using pointer/linked list.	Write a C program that uses functions to implement Queue (its operations) using pointer/linked list.